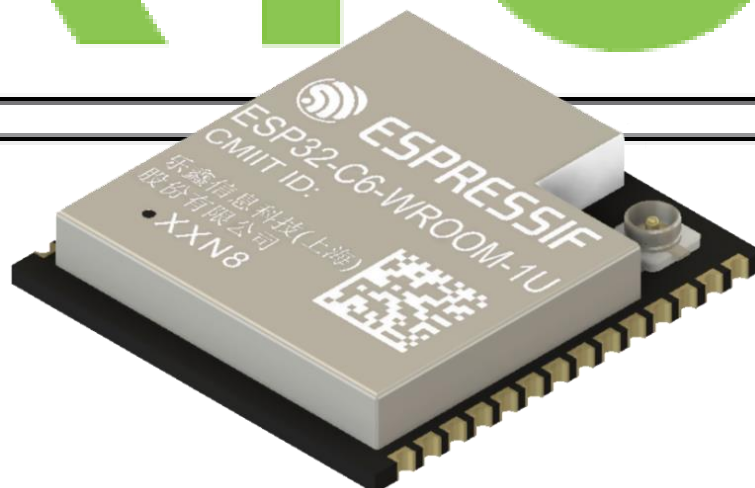


30-Day FreeRTOS Course for ESP32 Using ESP-IDF

(Day 12)

**“Mutexes and
Recursive Mutexes”**

freeRTOS



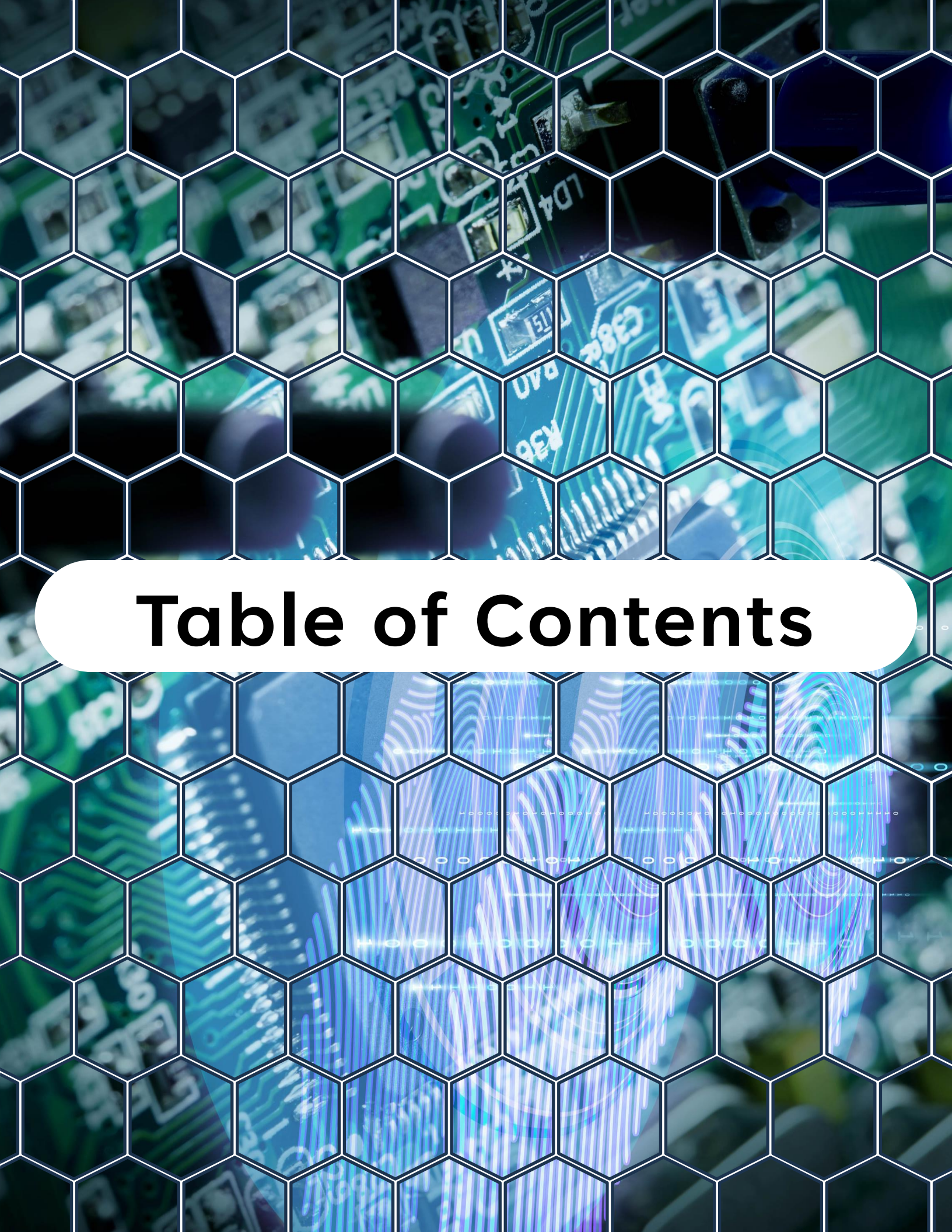
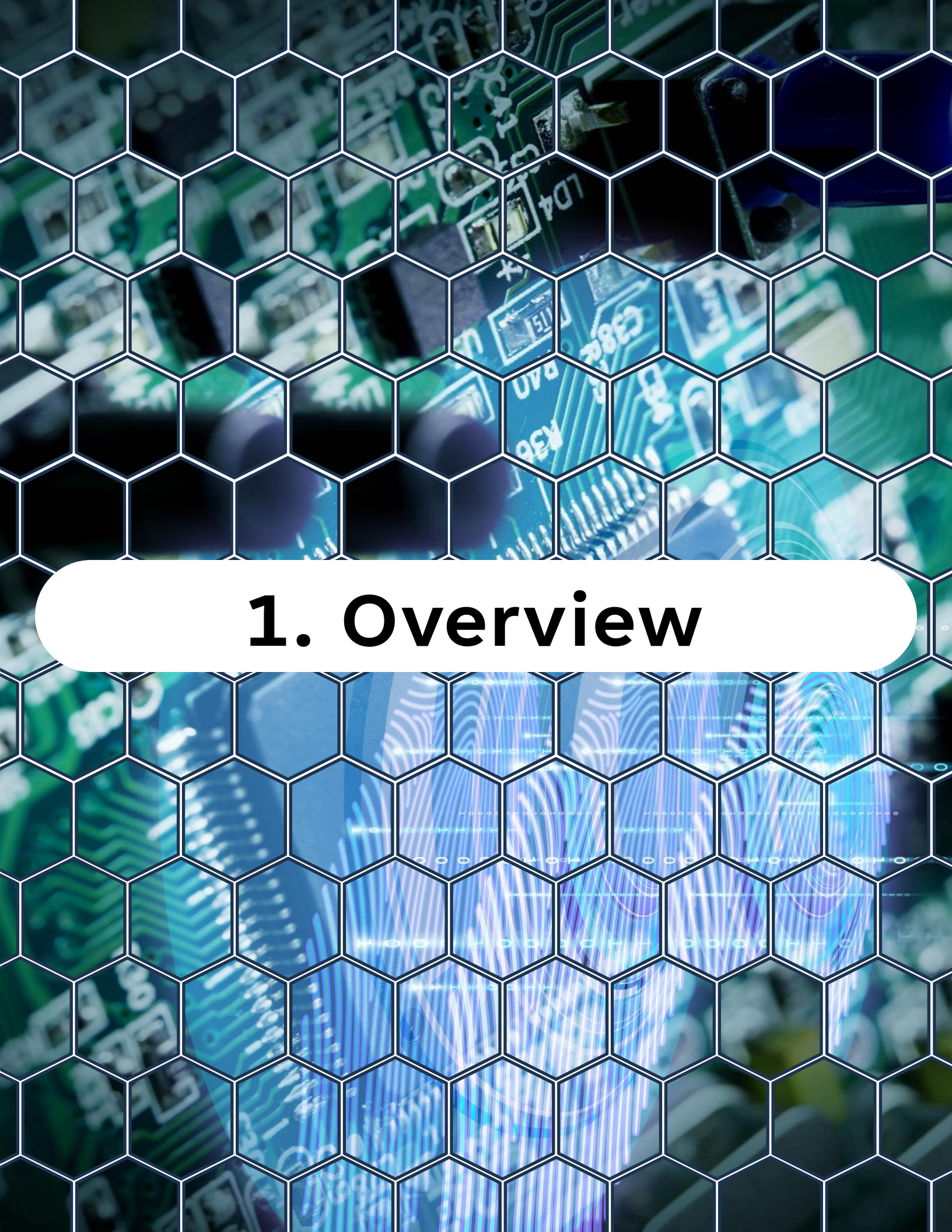


Table of Contents

Table of Contents

1. Overview
2. What Is a Mutex?
3. Mutex vs Binary Semaphore
4. Priority Inversion Problem
5. Recursive Mutexes
6. Example – Protecting UART with a Mutex
7. Best Practices
8. Summary
9. Challenge for Today



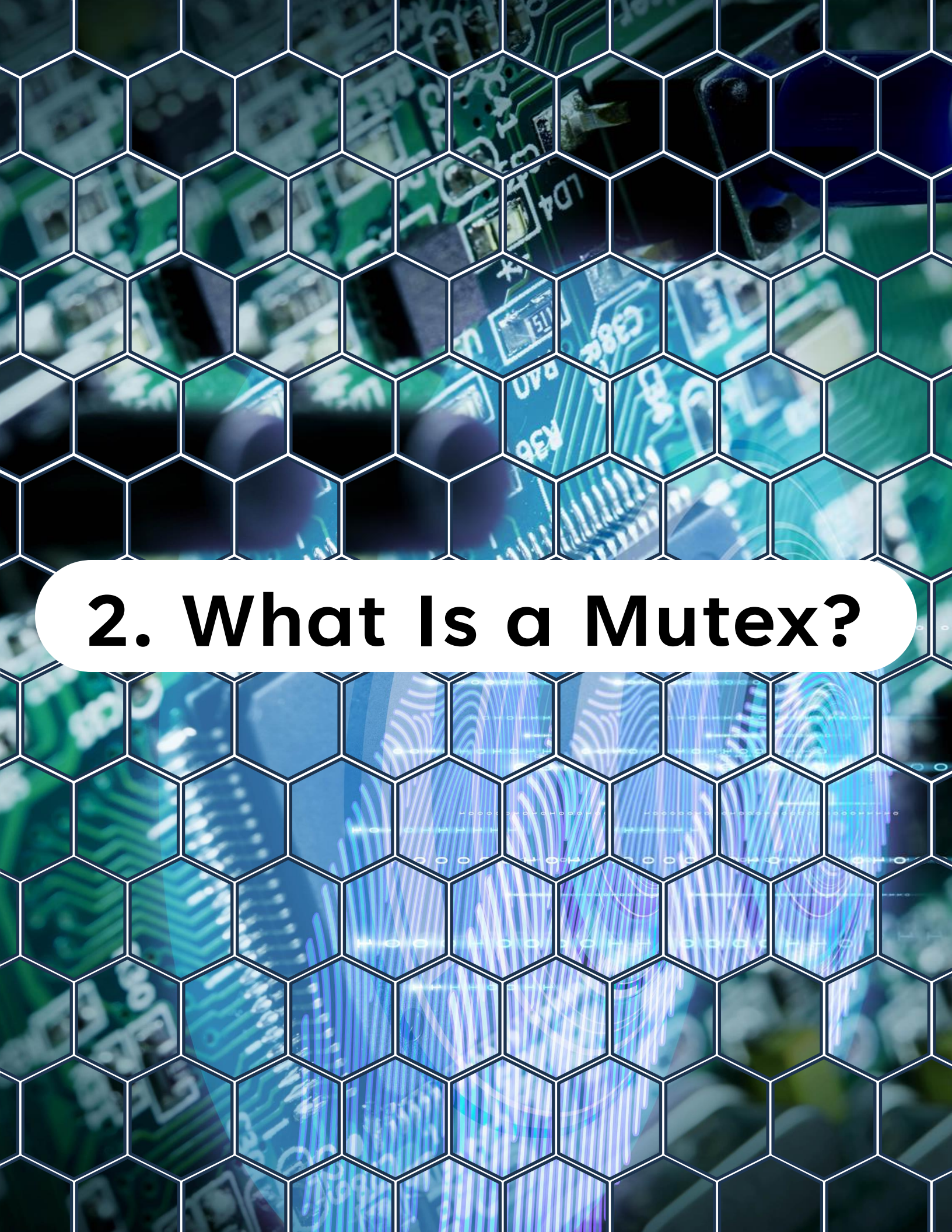
1. Overview

1. Overview

On **Day 12**, you'll learn:

- What mutexes are in FreeRTOS
- How they differ from binary semaphores
- The problem of priority inversion and how FreeRTOS solves it
- What recursive mutexes are and when to use them
- A practical ESP32 example: protecting a shared UART resource
- Best practices for mutex usage

By the end of this lesson, you'll understand how to use mutexes to safely share resources between tasks.



2. What Is a Mutex?

2. What Is a Mutex?

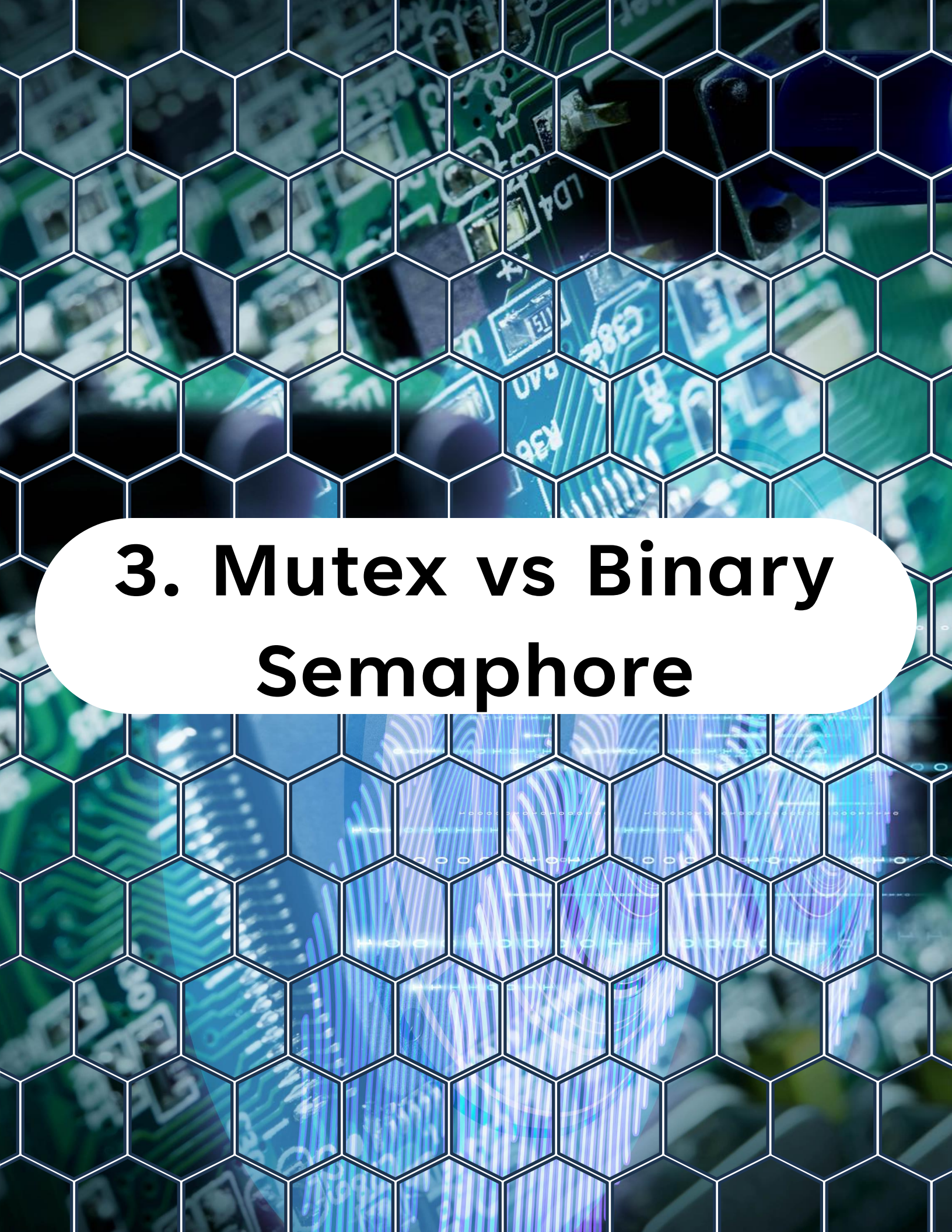
A **mutex (MUTual EXclusion)** is a synchronization primitive used to protect access to shared resources. Only one task can hold the mutex at a time:

- **Locked (taken)** → resource is in use
- **Unlocked (available)** → resource is free



Key feature:

FreeRTOS mutexes include priority inheritance, which helps prevent priority inversion.



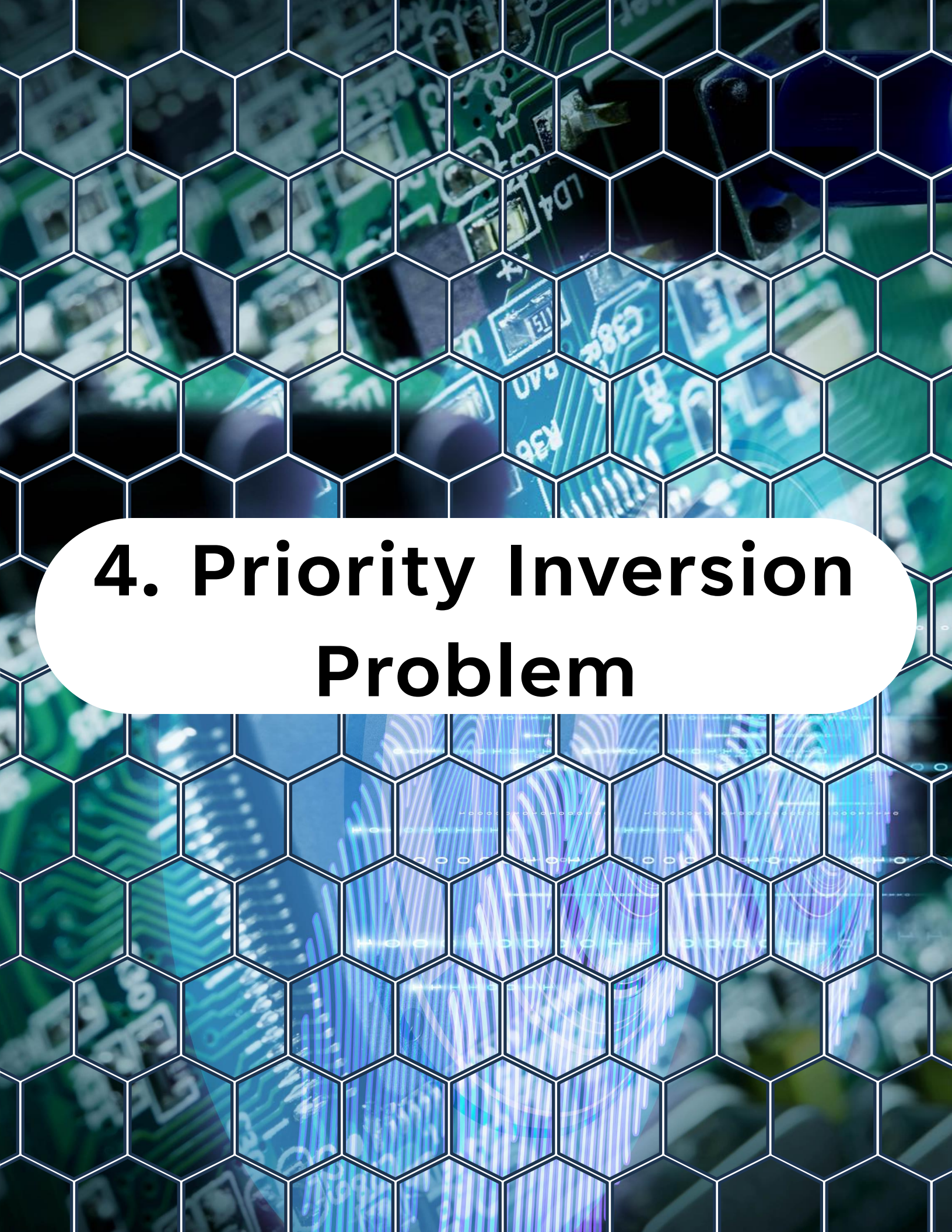
3. Mutex vs Binary Semaphore

3. Mutex vs Binary Semaphore

Feature	Mutex	Binary Semaphore
Purpose	Protect shared resources	Event signaling
Ownership	Task that takes must give	No ownership rules
Priority Inheritance	Yes	No
ISR Usage	Not ISR-safe	ISR-safe



Use a **mutex** for **resource protection**, and a **binary semaphore** for **event signaling**.



4. Priority Inversion Problem

4. Priority Inversion Problem

Priority inversion happens when:

A **low-priority task** holds a resource (mutex).

A **high-priority task** is blocked waiting for it.

A **medium-priority task** preempts the low-priority one.

Result:

the **high-priority task starves** until the medium-priority task finishes.



Solution:

FreeRTOS automatically **temporarily boosts the priority** of the low-priority task holding the mutex → allows it to release the resource quickly.



5. Recursive Mutexes

5. Recursive Mutexes

A **recursive mutex** is a special type of mutex that allows the same task to lock the mutex multiple times without deadlocking, as long as it releases it the same number of times.

Useful for:

- Functions that call other functions using the same shared resource.
- Complex APIs where nested calls may need the same lock.

Create with:

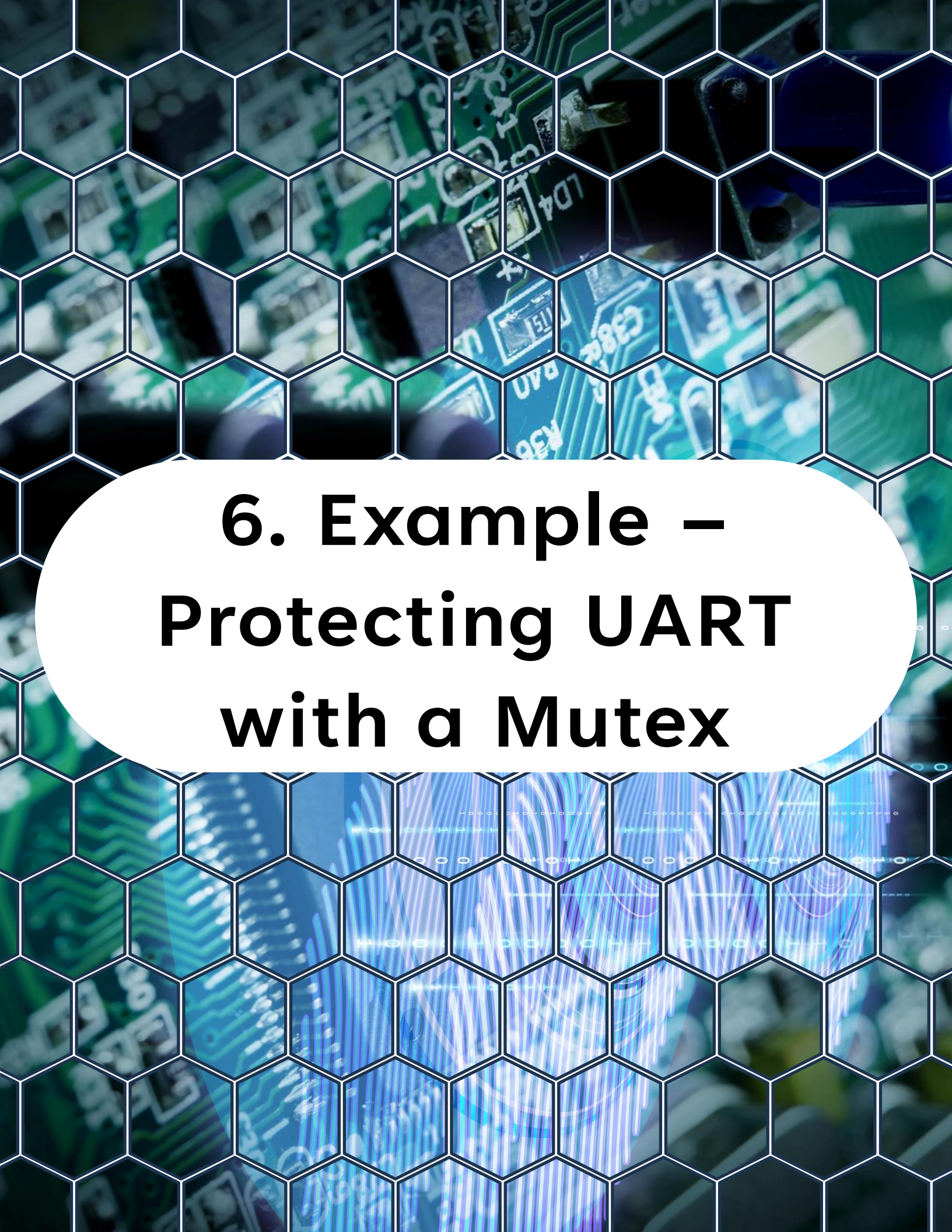


```
1 SemaphoreHandle_t xSemaphoreCreateRecursiveMutex(void);
```

Take/Give with:



```
1 xSemaphoreTakeRecursive(mutex, timeout);  
2 xSemaphoreGiveRecursive(mutex);
```

The background of the slide features a close-up, slightly blurred image of a green printed circuit board (PCB). The board is populated with various electronic components, including integrated circuits and surface-mount components. A white hexagonal grid pattern is overlaid on the entire image, creating a honeycomb effect. In the center, a white rounded rectangle contains the title text in bold black font.

6. Example – Protecting UART with a Mutex

6. Example – Protecting UART with a Mutex

Imagine two tasks writing to UART. Without a mutex, their messages may interleave and become unreadable.

Code Example:



```
1  #include <stdio.h>
2  #include "freertos/FreeRTOS.h"
3  #include "freertos/task.h"
4  #include "freertos/semphr.h"
5  #include "driver/uart.h"
6
7  #define TAG "DAY12"
8  #define UART_PORT UART_NUM_0
9
10 SemaphoreHandle_t uart_mutex;
11
12 // Task function that simulates writes to UART
13 void uart_task(void *pvParameter) {
14     // Get the task name, passed as parameter
15     const char *task_name = (const char *) pvParameter;
16
17     while (1) {
18         // Try to take the mutex before accessing UART
19         if (xSemaphoreTake(uart_mutex, portMAX_DELAY)) {
20             for (int i = 0; i < 5; i++) {
21                 printf("%s writing line %d\n", task_name, i);
22                 vTaskDelay(pdMS_TO_TICKS(200));
23             }
24             xSemaphoreGive(uart_mutex);
25         }
26     }
27 }
```

6. Example – Protecting UART with a Mutex

```
11
12 // Task function that simulates writes to UART
13 void uart_task(void *pvParameter) {
14     // Get the task name, passed as parameter
15     const char *task_name = (const char *) pvParameter;
16
17     while (1) {
18         // Try to take the mutex before accessing UART
19         if (xSemaphoreTake(uart_mutex, portMAX_DELAY)) {
20             for (int i = 0; i < 5; i++) {
21                 printf("%s writing line %d\n", task_name, i);
22                 vTaskDelay(pdMS_TO_TICKS(200));
23             }
24             // Release the mutex
25             xSemaphoreGive(uart_mutex);
26         }
27         // Delay before next attempt
28         vTaskDelay(pdMS_TO_TICKS(1000));
29     }
30 }
31
32 void app_main() {
33     // Create mutex
34     uart_mutex = xSemaphoreCreateMutex();
35     if (uart_mutex == NULL) {
36         printf("Failed to create mutex\n");
37         return;
38     }
39
40     // Create two tasks sharing UART
41     xTaskCreate(uart_task, "TaskA", 2048, "Task A", 5, NULL);
42     xTaskCreate(uart_task, "TaskB", 2048, "Task B", 5, NULL);
43 }
44
```


6. Example – Protecting UART with a Mutex

Expected Behavior

Without a mutex:

Task A writing line 0

Task B writing line 0

Task A writing line 1

Task B writing line 1

(interleaved, messy)

With a mutex:

Task A writing line 0

Task A writing line 1

...

Task B writing line 0

Task B writing line 1

...

Each task writes its block without interference.



7. Best Practices

7. Best Practices

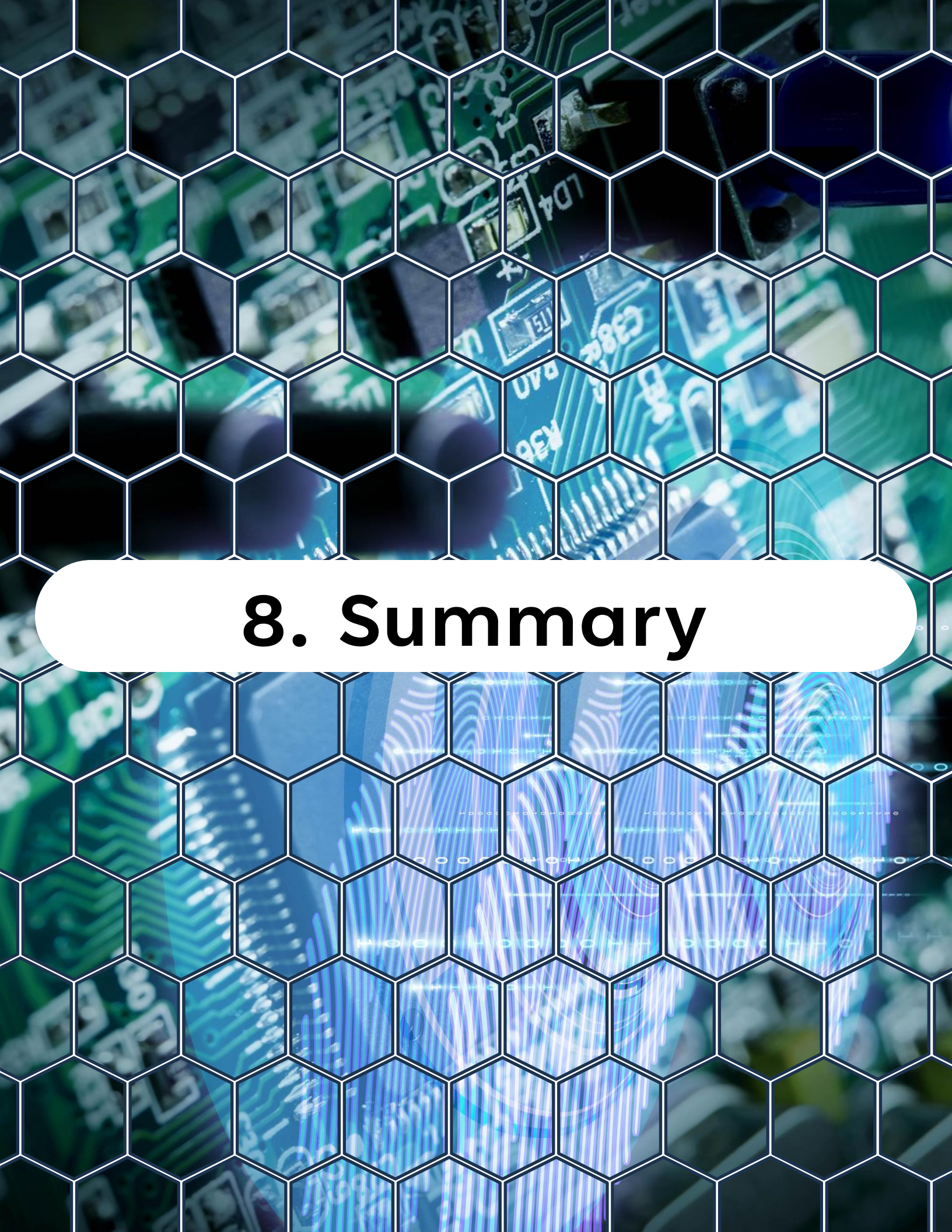
✓ Use **mutexes** only for **resource protection** (not for events).

✓ Keep **critical sections short** to avoid blocking other tasks.

✓ Avoid giving a mutex from an ISR — use semaphores for ISR signaling instead.

✓ Use **recursive mutexes** only when nested locking is unavoidable.

✓ Monitor for deadlocks — ensure every **take** has a matching **give**.



8. Summary

8. Summary

- ✓ Mutexes are for mutual exclusion when sharing resources.
- ✓ They include priority inheritance to solve priority inversion.
- ✓ Recursive mutexes allow the **same task to lock multiple times**.
- ✓ Always use `xSemaphoreGive()` after `xSemaphoreTake()`.



9. Challenge for Today

9. Challenge for Today

Extend the example by:

1. Adding a **third task** that logs timestamps to UART.
2. Observe how the mutex ensures only one task at a time writes clean output.
3. Try replacing the mutex with a binary semaphore — notice the missing priority inheritance protection.

"Thanks for watching!

If you enjoyed this video,

*make sure to hit the like button and
subscribe to stay updated with my latest
content.*

*Don't forget to check out my other
videos for more tips and tutorials on
Embedded C, Python, hardware designs,
etc. Keep exploring, keep learning, and
I'll see you in the next video!"*



<https://www.linkedin.com/in/yamil-garcia>

<https://www.youtube.com/@LearningByTutorials>

<https://github.com/god233012yamil>